

Data Pipeline Specification (SR)

Modul: data_pipeline

Jezik: SR

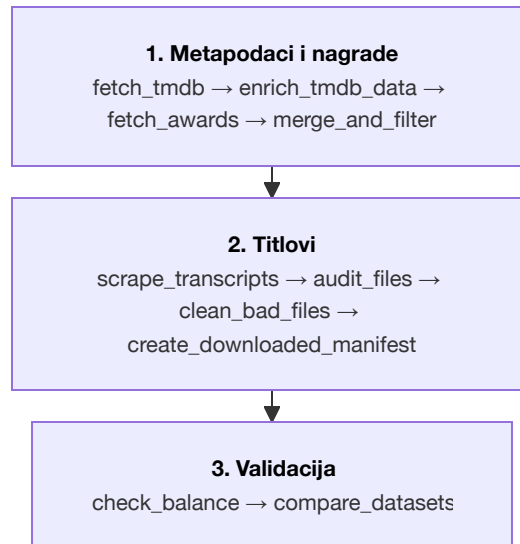
Sadržaj

- [1. Pregled pipeline-a](#)
- [2. Skripte](#)
 - [2.1 fetch_tmdb.py](#)
 - [2.2 enrich_tmdb_data.py](#)
 - [2.3 fetch_awards.py](#)
 - [2.4 merge_and_filter.py](#)
 - [2.5 scrape_transcripts.py](#)
 - [2.6 audit_files.py](#)
 - [2.7 clean_bad_files.py](#)
 - [2.8 create_downloaded_manifest.py](#)
 - [2.9 check_balance.py](#)
 - [2.10 compare_datasets.py](#)

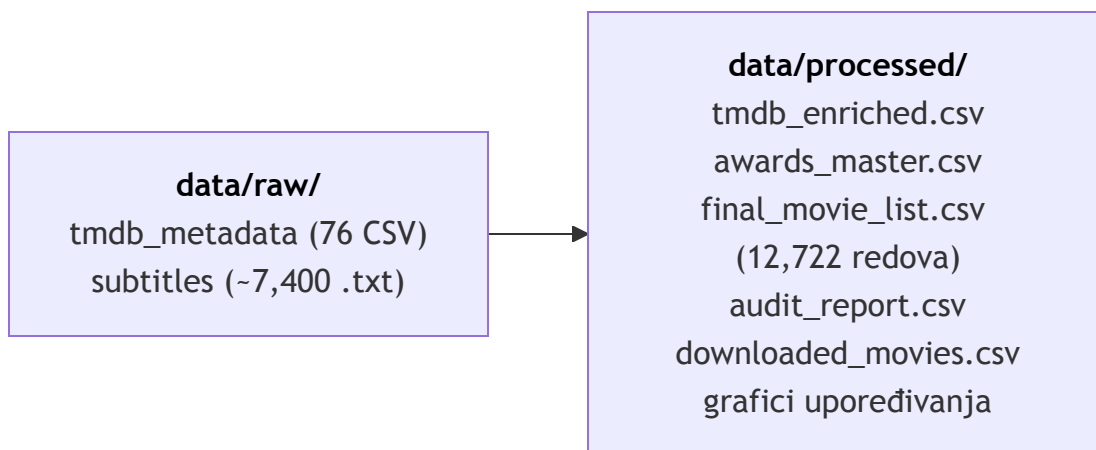
Ovaj modul sadrži 10 skripti koje se pokreću sekvencijalno kako bi izgradile kompletan filmski dataset od nule. Svaka skripta obrađuje jednu fazu — od preuzimanja sirovih API podataka do pravljenja čistog dataseta spremnog za analizu.

1. Pregled pipeline-a

Sekvencijalni tok (10 skripti):



Artefakti koje pipeline proizvodi:



2. Skripte

2.1 `fetch_tmdb.py` — Preuzimanje metapodataka o filmovima

Kreira osnovu baze sakupljajući filmove iz poslednjih 70 godina. Koristi API pozive da sistematski pretraži TMDB i izvuče 500 top filmova po godini. Na godisnjoj bazi cuva CSV checkpoint-e kako bi podaci bili bezbedni u slucaju da skripta pukne na pola posla.

Ulaz: TMDB API kljuc (`.env`)

Izlaz: `data/raw/tmdb_metadata/movies_YYYY.csv` (76 fajlova), `data/processed/tmdb_all_raw.csv` (konsolidovano)

2.2 `enrich_tmdb_data.py` — Dodavanje finansijskih detalja

Sluzi za preuzimanje konkretnih finansijskih metrika (budget i revenue) koje nam trebaju za racunanje ROI-ja (nas target za Market Success). Uzima osnovnu listu i gadjja TMDB API za te detalje i jako bitne IMDb ID-jeve. Pametno radi batch processing i ima crash recovery — automatski pauzira kad udarimo u API rate limite kako nas ne bi blokirali.

Ulaz: `data/processed/tmdb_all_raw.csv`

Izlaz: `data/processed/tmdb_enriched.csv`

2.3 `fetch_awards.py` — Prikupljanje podataka o nagradama

Cilj je da napravi "ground truth" listu za nas Critical Success model. Preko zvanicnog Wikipedia API-ja skida podatke za glavne filmske nagrade (Oskari, BAFTA, Zlatni globus, itd.). Parsira potpuno haoticne HTML tabele, cisti naslove i pakuje sve to u jedan cist CSV spisak nagrađivanih filmova.

Ulaz: Wikipedia API

Izlaz: `data/processed/awards_master.csv`

2.4 merge_and_filter.py — Spajanje i filtriranje

Radi kao "Great Filter" da ocisti dataset od irelevantnih filmova i djubreta. Spaja TMDB metapodatke, finansije i Wikipedia nagrade u jednu masivnu master tabelu. Strogo propusta samo filmove koji prolaze barem jedan od tri uslova: imaju validne finansijske podatke, osvojili su neku veliku nagradu ili imaju gomilu glasova (dokaz da je pravi film, a ne neki studentski projekat).

Ulaz: data/processed/tmdb_all_raw.csv , data/processed/tmdb_enriched.csv ,
data/processed/awards_master.csv

Izlaz: data/processed/final_movie_list.csv (12.722 filma)

2.5 `scrape_transcripts.py` — Preuzimanje tekstova titlova

Glavni alat za nabavljanje samog teksta (dijaloga) koji ce nas NLP model zapravo da zvace. Glumi pravi web browser (menja nasumicne User-Agent-e) da bi zaobisao bot zastite i skrejpovao titlove direktno sa neta. Koristi fuzzy string matching logiku da bi bio apsolutno siguran da naslov filma koji skidamo zapravo odgovara onom iz nase baze.

Ulaz: `data/processed/final_movie_list.csv`

Izlaz: `data/raw/subtitles/{tmdb_id}.txt` (~7.400 fajlova)

2.6 `audit_files.py` — Provera kvaliteta titlova

Pre nego sto ubacimo tekst u ML model, ova skripta radi provere da li je neki fajl korumpiran tokom skidanja. Skenira ceo folder sa skinutim fajlovima i trazi skrivene greske. Flaguje fajlove koji su sumnjivo mali (ispod 2KB) ili u sebi greskom sadrze HTML error poruke tipa "404 Not Found" ili "Cloudflare block".

Ulaz: `data/raw/subtitles/`

Izlaz: `data/processed/audit_report.csv`

2.7 `clean_bad_files.py` — Brisanje korumpiranih fajlova

Fizicki brise lose fajlove koje je audit skripta nasla, da nam ne bi upropastili NLP procesiranje vokabulara. Cita audit report i direktno brise te fajlove sa hard diska. Ima ugradjen safety check — trazi da rukno u terminalu ukucamo potvrdu ("DELETE") pre nego sto bilo sta trajno obrise, cisto da ne obrisemo sve slucajno.

Ulaz: `data/processed/audit_report.csv`

Izlaz: Korumpirani fajlovi obrisani iz `data/raw/subtitles/`

2.8 create_downloaded_manifest.py — Pravljenje finalnog manifesta

Nakon ciscenja, proverava sta nam je zapravo ostalo od validnih i upotrebljivih podataka. Upoređuje tekstualne fajlove koji fizicki postoje na disku sa nasom filtriranom master bazom. Pravi finalni "Source of Truth" CSV manifest, sto garantuje da svaki red u datasetu za treniranje ima svoj pripadajuci validan .txt fajl.

Ulaz: data/raw/subtitles/ , data/processed/final_movie_list.csv

Izlaz: data/processed/downloaded_movies.csv

2.9 check_balance.py — Provera balansa klasa

Proverava da li je nas finalni dataset statisticki okej i spreman za Machine Learning fazu. Analizira class imbalance za nase target varijable (npr. gleda koliko zapravo imamo dobitnika nagrada naspram onih koji nisu dobili). Sluzi kao rana dijagnostika koja nam javlja ako cemo morati da koristimo SMOTE balansiranje klasa pre samog treniranja modela.

Ulaz: data/processed/final_movie_list.csv , data/processed/audit_report.csv

Izlaz: Konzolni izvestaj sa distribucijom klasa

2.10 compare_datasets.py — Analiza pristranosti pri prikupljanju

Vizuelno proverava da li smo web scrapingom slucajno uneli neki bias u podatke. Upoređuje listu filmova koje smo prvobitno targetirali sa onima koje smo na kraju uspesno skinuli. Automatski generise i cuva pie i bar chart grafike (spremne za projektnu dokumentaciju) koji nam pokazuju da li se skrejper mozda vise mucio sa starijim filmovima ili specifcnim zanrovima.

Ulaz: data/processed/final_movie_list.csv , data/processed/downloaded_movies.csv

Izlaz: Grafikoni uporedjivanja (sačuvani u reports/figures/)